

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	Question	Bloom's Level
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

Code of Conduct:

*I Lokesh S (192511037) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.

Answer

Introduction

A university database stores information about students, courses, departments, and faculty members. If all these details are stored in a single table, the same information is repeated multiple times. This creates redundancy, wastes storage space, and leads to update, insertion, and deletion anomalies. Normalization is used to organize the database efficiently by dividing the data into multiple related tables while preserving data integrity.

Original Table

StudentID	StudentName	Department	CourseID	CourseName	FacultyName	FacultyPhone
101	Rahul	CSE	CS301	DBMS	Dr. Kumar	9876543210
101	Rahul	CSE	CS302	Operating Systems	Dr. Meena	9876543211
102	Priya	ECE	EC201	Signals	Dr. Ravi	9876543212
102	Priya	ECE	CS301	DBMS	Dr. Kumar	9876543210

Normalization Issues

1. Data Redundancy

Student details and faculty details are repeated for every course a student enrolls in. This increases storage requirements and makes maintenance difficult.

2. Update Anomaly

If Dr. Kumar's phone number changes, it must be updated in every row where his details appear. Missing even one record results in inconsistent data.

3. Insertion Anomaly

A new course cannot be added until at least one student enrolls in it because the course information is stored together with student information.

4. Deletion Anomaly

If the last student enrolled in a course withdraws, deleting that record also removes the course and faculty information from the database.

Functional Dependencies

- StudentID → StudentName, Department

- CourseID → CourseName, FacultyName
- FacultyName → FacultyPhone

The composite key (**StudentID, CourseID**) uniquely identifies each record because a student can enroll in multiple courses.

Conversion to First Normal Form (1NF)

A relation is in **1NF** if all attributes contain atomic values and there are no repeating groups.

The given table already satisfies 1NF because every column contains a single value. However, redundancy still exists.

Conversion to Second Normal Form (2NF)

A relation is in **2NF** if it is in 1NF and has no partial dependency on the composite key.

The following tables are created:

Student Table

StudentID	StudentName	Department
101	Rahul	CSE
102	Priya	ECE

Course Table

CourseID	CourseName	FacultyName
CS301	DBMS	Dr. Kumar
CS302	Operating Systems	Dr. Meena
EC201	Signals	Dr. Ravi

Enrollment Table

StudentID	CourseID
101	CS301
101	CS302
102	EC201
102	CS301

Now, student details depend only on **StudentID**, and course details depend only on **CourseID**.

Conversion to Third Normal Form (3NF)

The Course table still contains a transitive dependency because:

FacultyName → **FacultyPhone**

To remove this dependency, create a separate Faculty table.

Faculty Table

FacultyID	FacultyName	FacultyPhone
F101	Dr. Kumar	9876543210
F102	Dr. Meena	9876543211
F103	Dr. Ravi	9876543212

Modified Course Table

CourseID	CourseName	FacultyID
CS301	DBMS	F101
CS302	Operating Systems	F102
EC201	Signals	F103

Final 3NF Schema

- **Student(StudentID, StudentName, Department)**
- **Faculty(FacultyID, FacultyName, FacultyPhone)**
- **Course(CourseID, CourseName, FacultyID)**
- **Enrollment(StudentID, CourseID)**

Advantages of 3NF

- Eliminates duplicate data.
- Removes update, insertion, and deletion anomalies.
- Improves data consistency and integrity.
- Makes database maintenance easier.
- Increases storage efficiency and query performance.

Conclusion

The original university table contained repeated student and faculty information, resulting in redundancy and anomalies. By applying **1NF, 2NF, and 3NF**, the database is decomposed into four separate relations: **Student, Faculty, Course, and Enrollment**. The final design is well-

structured, minimizes redundancy, and ensures efficient data management, making it suitable for a university database system.

Q2. A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach.

Answer

Introduction

In a company, an employee may work in one or more departments. To manage this relationship efficiently, the database contains separate tables for employees, departments, and employee-department assignments. SQL joins and aggregate functions can be used to identify employees who are assigned to multiple departments.

Database Schema

Employee Table

EmpID	EmpName	Designation
101	Rahul	Developer
102	Priya	Tester
103	Arun	Manager

Department Table

DeptID	DeptName
D01	HR
D02	IT
D03	Finance

Employee_Department Table

EmpID	DeptID
101	D01
101	D02
102	D02
103	D01

EmpID	DeptID
103	D03

SQL Query

```
mysql> SELECT
->     e.EmpID,
->     e.EmpName,
->     COUNT(ed.DeptID) AS TotalDepartments
-> FROM Employee e
-> INNER JOIN Employee_Department ed
-> ON e.EmpID = ed.EmpID
-> GROUP BY e.EmpID, e.EmpName
-> HAVING COUNT(ed.DeptID) > 1;
+-----+-----+-----+
| EmpID | EmpName | TotalDepartments |
+-----+-----+-----+
| 101   | Rahul   | 2                |
| 103   | Arun    | 2                |
+-----+-----+-----+
2 rows in set (0.01 sec)
```

Justification of the Approach

- **JOIN** combines the Employee and Employee_Department tables using the common attribute EmpID.
- **GROUP BY** groups all department records belonging to the same employee.
- **COUNT()** counts the number of departments assigned to each employee.
- **HAVING COUNT(DeptID) > 1** filters only those employees who are associated with multiple departments.
- This approach is efficient, easy to understand, and avoids redundant data retrieval.

Advantages

- Retrieves only the required employees.
- Works efficiently for large databases.
- Uses SQL aggregate functions effectively.
- Can be extended to retrieve department names or employee details.

Conclusion

The SQL query uses **JOIN**, **GROUP BY**, and **HAVING** clauses to identify employees working in multiple departments. This approach efficiently retrieves the required information

while maintaining data integrity and reducing query complexity. It is suitable for real-world company database systems.

Q3. Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

Answer

Introduction

Boyce-Codd Normal Form (BCNF) is a stronger version of Third Normal Form (3NF). A relation is in BCNF if, for every functional dependency $X \rightarrow Y$, **X must be a super key**. If this condition is not satisfied, the relation should be decomposed into smaller relations to eliminate redundancy and anomalies.

Given Relation

Consider the relation:

$R(A, B, C, D)$

Functional Dependencies (FDs)

$A \rightarrow B$

$B \rightarrow C$

$A \rightarrow D$

Step 1: Determine the Candidate Key

To find the candidate key, calculate the closure of attribute **A**.

Closure of A

A^+

$= \{A\}$

Using $A \rightarrow B$

$A^+ = \{A, B\}$

Using $B \rightarrow C$

$A^+ = \{A, B, C\}$

Using $A \rightarrow D$

$A^+ = \{A, B, C, D\}$

Since $A^+ = \{A, B, C, D\}$, it contains all attributes of the relation.

Therefore, the Candidate Key is:

Candidate Key = A

Step 2: Check BCNF

BCNF Rule:

Every determinant in a functional dependency must be a candidate key (or super key).

Check each Functional Dependency

1.

$A \rightarrow B$

- Determinant = A
- A is the candidate key.

Satisfies BCNF

2.

$A \rightarrow D$

- Determinant = A
- A is the candidate key.

Satisfies BCNF

3.

$B \rightarrow C$

- Determinant = B
- B is **not** a candidate key.

Violates BCNF

Hence, the relation **R(A, B, C, D)** is **not in BCNF**.

Step 3: BCNF Decomposition

Since **B $\rightarrow$ C** violates BCNF, decompose the relation into two relations.

Relation 1

R1(B, C)

Functional Dependency:

$B \rightarrow C$

Here, **B** is the candidate key of **R1**, so **R1** satisfies BCNF.

Relation 2

$R_2(A, B, D)$

Functional Dependencies:

$A \rightarrow B$

$A \rightarrow D$

Here, **A** is the candidate key of **R2**, so **R2** also satisfies BCNF.

Final BCNF Relations

R1

B C

B1 C1

B2 C2

R2

A B D

A1 B1 D1

A2 B2 D2

Advantages of BCNF Decomposition

- Eliminates redundancy caused by functional dependencies.
- Removes update, insertion, and deletion anomalies.
- Ensures better data consistency.
- Improves database integrity.
- Produces a well-structured relational database.

Conclusion

The candidate key of the given relation is **A**. Although the dependencies $A \rightarrow B$ and $A \rightarrow D$ satisfy BCNF, the dependency $B \rightarrow C$ violates BCNF because **B** is not a candidate key. Therefore, the relation is decomposed into **R1(B, C)** and **R2(A, B, D)**, both of which satisfy BCNF. This decomposition reduces redundancy and improves the overall efficiency and consistency of the database.

Q4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

Introduction

In a banking system, different users such as **Managers, Employees, and Customers** require different levels of access to the database. Sensitive information like account balances and customer details must be protected from unauthorized access. This can be achieved using **SQL Views** and **Integrity Constraints**.

Bank Account Table

```
mysql> CREATE TABLE Account (  
-> AccountNo INT PRIMARY KEY,  
-> CustomerName VARCHAR(50) NOT NULL,  
-> Branch VARCHAR(50),  
-> Balance DECIMAL(10,2) CHECK (Balance >= 1000),  
-> AccountType VARCHAR(20)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

Insert Sample Records

```
mysql> INSERT INTO Account VALUES  
-> (1001, 'Rahul', 'Chennai', 50000, 'Savings'),  
-> (1002, 'Priya', 'Bangalore', 30000, 'Current'),  
-> (1003, 'Arun', 'Chennai', 45000, 'Savings');  
Query OK, 3 rows affected (0.02 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select * from account;
```

AccountNo	CustomerName	Branch	Balance	AccountType
1001	Rahul	Chennai	50000.00	Savings
1002	Priya	Bangalore	30000.00	Current
1003	Arun	Chennai	45000.00	Savings

3 rows in set (0.00 sec)

Step 1: Create Views

(a) Manager View

Managers can access all account details.

```
mysql> CREATE VIEW Manager_View AS  
-> SELECT *  
-> FROM Account;  
Query OK, 0 rows affected (0.02 sec)
```

(b) Employee View

Employees can view customer details but cannot view account balance.

```
mysql> CREATE VIEW Employee_View AS
-> SELECT AccountNo,
->         CustomerName,
->         Branch,
->         AccountType
-> FROM Account;
Query OK, 0 rows affected (0.02 sec)
```

(c) Customer View

Customers can view only their own account information.

```
mysql> CREATE VIEW Customer_View AS
-> SELECT AccountNo,
->         CustomerName,
->         Balance
-> FROM Account;
Query OK, 0 rows affected (0.02 sec)
```

Step 2: Apply Integrity Constraints

Primary Key

Ensures each account number is unique.

PRIMARY KEY(AccountNo)

NOT NULL Constraint

Customer name cannot be empty.

CustomerName VARCHAR(50) NOT NULL

CHECK Constraint

Minimum account balance should be ₹1000.

CHECK(Balance >= 1000)

UNIQUE Constraint

To avoid duplicate account numbers or emails (if available).

UNIQUE(AccountNo)

FOREIGN KEY Constraint

If a Branch table exists, link the branch information.

```
mysql> CREATE TABLE Branch (
->     BranchID INT PRIMARY KEY,
->     BranchName VARCHAR(50)
-> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE Account (  
->     AccountNo INT PRIMARY KEY,  
->     CustomerName VARCHAR(50),  
->     BranchID INT,  
->     Balance DECIMAL(10,2),  
->     FOREIGN KEY (BranchID)  
->     REFERENCES Branch(BranchID)  
-> );  
Query OK, 0 rows affected (0.07 sec)
```

Advantages

- Views restrict unauthorized access to sensitive data.
- Managers can access complete account details.
- Employees can perform daily operations without viewing confidential balances.
- Constraints ensure valid and consistent data.
- Prevents duplicate records and invalid entries.

Conclusion

Using **SQL Views** and **Integrity Constraints**, the banking system provides secure data access while maintaining data integrity. Views ensure that each user can access only the required information, whereas constraints such as **PRIMARY KEY**, **NOT NULL**, **CHECK**, **UNIQUE**, and **FOREIGN KEY** maintain the accuracy, consistency, and reliability of the database.

Q5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

Answer

Introduction

A sales database stores information about customers, products, suppliers, and orders. If all these details are stored in a single table, duplicate data is created, leading to update, insertion, and deletion anomalies. Normalization is used to eliminate redundancy and improve data consistency.

Unnormalized Sales Table

Order ID	CustomerID	CustomerName	ProductID	ProductName	SupplierID	SupplierName	SupplierPhone
O101	C101	Rahul	P101	Laptop	S101	Dell	9876543210
O102	C101	Rahul	P102	Mouse	S102	Logitech	9876543211
O103	C102	Priya	P101	Laptop	S101	Dell	9876543210

Problems Identified

- **Data Redundancy:** Customer and supplier details are repeated in multiple rows.
- **Update Anomaly:** Changing a supplier's phone number requires updating several records.
- **Insertion Anomaly:** A new supplier cannot be added until a product or order exists.
- **Deletion Anomaly:** Deleting the last order of a supplier removes the supplier information.

Functional Dependencies

- CustomerID → CustomerName
- ProductID → ProductName, SupplierID
- SupplierID → SupplierName, SupplierPhone
- OrderID → CustomerID, ProductID

First Normal Form (1NF)

The table is in **1NF** because all attributes contain atomic values.

Second Normal Form (2NF)

Remove partial dependencies by creating separate tables.

Customer Table

CustomerID	CustomerName
C101	Rahul
C102	Priya

Supplier Table

SupplierID	SupplierName	SupplierPhone
S101	Dell	9876543210
S102	Logitech	9876543211

Product Table

ProductID	ProductName	SupplierID
P101	Laptop	S101
P102	Mouse	S102

Orders Table

OrderID	CustomerID	ProductID
O101	C101	P101
O102	C101	P102
O103	C102	P101

Third Normal Form (3NF)

The transitive dependency:

ProductID → SupplierID → SupplierName, SupplierPhone

is removed by storing supplier information in a separate **Supplier** table.

Final 3NF Schema

Customer(CustomerID, CustomerName)

Supplier(SupplierID, SupplierName, SupplierPhone)

Product(ProductID, ProductName, SupplierID)

Orders(OrderID, CustomerID, ProductID)

Advantages of Normalization

- Eliminates duplicate data.
- Reduces update, insertion, and deletion anomalies.
- Improves data integrity and consistency.

- Makes database maintenance easier.
- Optimizes storage and query performance.

Conclusion

The original sales database contained redundant customer and supplier information, resulting in various anomalies. By applying **1NF, 2NF, and 3NF**, the data is divided into **Customer, Supplier, Product, and Orders** tables. This normalized design removes redundancy, improves data integrity, and provides an efficient database structure for managing sales information.

Q6. Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Answer

Introduction

In a customer-order database, information is stored in separate **Customer** and **Orders** tables. To retrieve meaningful information such as **frequent customers**, SQL uses **JOIN** operations to combine data from multiple tables and **subqueries** to filter records based on specific conditions.

Customer Table

CustomerID	CustomerName	City
C101	Rahul	Chennai
C102	Priya	Bangalore
C103	Arun	Chennai
C104	Sneha	Hyderabad

Orders Table

OrderID	CustomerID	Product	Amount
O101	C101	Laptop	50000
O102	C101	Mouse	1000
O103	C101	Keyboard	2500

OrderID	CustomerID	Product	Amount
O104	C102	Mobile	25000
O105	C103	Laptop	50000
O106	C103	Printer	12000

Query 1: Display Customer Details with Their Orders (Using JOIN)

```
mysql> SELECT
->     c.CustomerID,
->     c.CustomerName,
->     c.City,
->     o.OrderID,
->     o.Product,
->     o.Amount
-> FROM Customer c
-> INNER JOIN Orders o
-> ON c.CustomerID = o.CustomerID;
```

CustomerID	CustomerName	City	OrderID	Product	Amount
101	Rahul	Chennai	1001	Laptop	55000.00
101	Rahul	Chennai	1002	Mouse	1200.00
101	Rahul	Chennai	1003	Keyboard	2500.00
102	Priya	Bangalore	1004	Monitor	15000.00
103	Arun	Hyderabad	1005	Printer	10000.00
103	Arun	Hyderabad	1006	Scanner	8000.00
104	Sneha	Chennai	1007	Laptop	60000.00

7 rows in set (0.00 sec)

Query 2: Find Frequent Customers (Using JOIN)

Customers having **more than 2 orders**.

```
mysql> SELECT
->     c.CustomerID,
->     c.CustomerName,
->     COUNT(o.OrderID) AS TotalOrders
-> FROM Customer c
-> INNER JOIN Orders o
-> ON c.CustomerID = o.CustomerID
-> GROUP BY c.CustomerID, c.CustomerName
-> HAVING COUNT(o.OrderID) > 2;
```

CustomerID	CustomerName	TotalOrders
101	Rahul	3

1 row in set (0.00 sec)

Query 3: Find Frequent Customers (Using Subquery)

```
mysql> SELECT CustomerID, CustomerName
-> FROM Customer
-> WHERE CustomerID IN
-> (
->     SELECT CustomerID
->     FROM Orders
->     GROUP BY CustomerID
->     HAVING COUNT(OrderID) > 2
-> );
```

CustomerID	CustomerName
101	Rahul

1 row in set (0.00 sec)

Justification

- **INNER JOIN** combines data from the **Customer** and **Orders** tables using the common attribute CustomerID.
- **GROUP BY** groups all orders belonging to the same customer.
- **COUNT()** calculates the total number of orders placed by each customer.
- **HAVING** filters only customers who have placed more than two orders.
- The **subquery** first identifies frequent customers and then retrieves their details from the Customer table.

Advantages

- Retrieves meaningful information from multiple tables.
- Identifies loyal or frequent customers.
- Uses efficient SQL features like JOIN, GROUP BY, HAVING, and Subqueries.
- Helps businesses analyze customer purchasing patterns.

Conclusion

The required information is retrieved using both **JOIN** and **Subquery** techniques. The JOIN combines customer and order data, while the subquery identifies customers who have placed more than two orders. This approach efficiently analyzes customer activity and helps businesses recognize frequent customers for better decision-making.

Q7. A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

Answer

Introduction

A relation is in **Fourth Normal Form (4NF)** if it is in **BCNF** and contains **no non-trivial multivalued dependencies (MVDs)**. A multivalued dependency occurs when one attribute determines multiple independent values of another attribute. Decomposing such relations into 4NF removes redundancy and prevents anomalies.

Given Relation

Suppose a university stores student, course, and hobby details in a single table.

StudentID	StudentName	Course	Hobby
101	Rahul	DBMS	Cricket
101	Rahul	DBMS	Music
101	Rahul	AI	Cricket
101	Rahul	AI	Music
102	Priya	Networks	Dance
102	Priya	Networks	Reading

Multivalued Dependencies

StudentID $\twoheadrightarrow$ Course

StudentID $\twoheadrightarrow$ Hobby

Here,

- A student can enroll in multiple courses.
- A student can have multiple hobbies.
- Courses and hobbies are independent of each other.

This creates unnecessary duplicate records.

Problems in the Relation

- **Data Redundancy:** Course and hobby information is repeated.
- **Insertion Anomaly:** Adding a new hobby requires repeating course details.
- **Deletion Anomaly:** Deleting a course may also remove hobby information.
- **Update Anomaly:** Updating a hobby requires modifying multiple rows.

Decomposition into 4NF

StudentCourse Table

Final 4NF Schema

Employee

(EmployeeID **PK**, EmployeeName)

Employee_Skill

(EmployeeID **FK**, Skill)

Primary Key: (**EmployeeID, Skill**)

Employee_Project

(EmployeeID **FK**, Project)

Primary Key: (**EmployeeID, Project**)

Why the Decomposition Satisfies 4NF

The decomposed relations satisfy **Fourth Normal Form** because:

- Each table contains only one independent multi-valued fact.
- No non-trivial multi-valued dependency exists.
- Every multi-valued dependency has a super key as its determinant.
- Redundant combinations of skills and projects are eliminated.

Advantages of 4NF

- Eliminates data redundancy.
- Removes multi-valued dependencies.
- Prevents insertion, update, and deletion anomalies.
- Improves data consistency.
- Reduces storage requirements.
- Simplifies database maintenance.
- Supports efficient query processing.
- Provides a scalable database design.

Justification

The original relation stored employee skills and projects together, causing repeated combinations of skills and projects. By decomposing the relation into **Employee**, **Employee_Skill**, and **Employee_Project**, each table stores only one type of independent information. This removes multi-valued dependencies, eliminates redundancy, and maintains

referential integrity using primary and foreign keys. The resulting schema satisfies **Fourth Normal Form (4NF)** and is suitable for efficient database management.

Conclusion

The original Employee relation contained **multi-valued dependencies**, resulting in redundant data and database anomalies. After decomposition into **Employee**, **Employee_Skill**, and **Employee_Project** tables, all independent multi-valued attributes are separated. The final schema satisfies **Fourth Normal Form (4NF)**, eliminates redundancy, improves data integrity, and provides an efficient, scalable, and maintainable database structure for real-world applications.

Q8. A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

Scenario Analysis

A database stores employee information, but the designer failed to apply appropriate integrity constraints. As a result, duplicate records, invalid values, missing information, and broken relationships exist in the database. These inconsistencies reduce data accuracy and reliability.

The objective is to evaluate the schema, identify the problems caused by improper constraints, and suggest suitable **integrity constraints** to ensure data consistency, accuracy, and reliability.

Existing Schema

Employee Table

EmployeeID	EmployeeName	DepartmentID	Salary	Email
101	Arun	1	50000	arun@gmail.com
101	Arun	2	-1000	arun@gmail.com
NULL	Priya	3	45000	priya@gmail.com
103	Rahul	8	60000	rahul@gmail.com

Department Table

DepartmentID	DepartmentName
1	HR
2	IT
3	Finance

Problems Identified

The schema contains several inconsistencies:

- Duplicate Employee IDs.
- NULL values in the primary key.
- Negative salary values.
- Invalid DepartmentID that does not exist.
- Duplicate email addresses.
- Missing validation rules.
- Weak referential integrity.

These problems occur because appropriate integrity constraints were not implemented.

Suggested Integrity Constraints

Primary Key Constraint

The **EmployeeID** must uniquely identify every employee and should never be NULL.

```
mysql> CREATE TABLE Employee
-> (
-> EmployeeID INT PRIMARY KEY,
-> EmployeeName VARCHAR(50),
-> DepartmentID INT,
-> Salary DECIMAL(10,2),
-> Email VARCHAR(100)
-> );
Query OK, 0 rows affected (0.06 sec)
```

Purpose

- Prevents duplicate employee records.
- Does not allow NULL values.
- Ensures every employee has a unique identity.

Foreign Key Constraint

The DepartmentID should reference an existing department.

```
mysql> FOREIGN KEY (DepartmentID)
-> REFERENCES Department(DepartmentID);
```

Purpose

- Prevents invalid department values.
- Maintains referential integrity.
- Ensures every employee belongs to a valid department.

NOT NULL Constraint

Mandatory fields should never be left empty.

```
mysql> EmployeeName VARCHAR (50) NOT NULL,
-> Salary DECIMAL (10,2) NOT NULL
```

Purpose

- Ensures essential information is always entered.
- Prevents incomplete records.

UNIQUE Constraint

Each employee email should be unique.

```
mysql> Email VARCHAR(100) UNIQUE|
```

Purpose

- Prevents duplicate email addresses.
- Maintains unique employee contact information.

CHECK Constraint

```
mysql> Salary should always be positive.
-> Salary DECIMAL(10,2)
-> CHECK (Salary > 0)
-> Another example:
-> CHECK (DepartmentID BETWEEN 1 AND 3)
```

Purpose

- Prevents invalid salary values.
- Ensures only valid department IDs are accepted.

DEFAULT Constraint

Assign a default salary if no salary is entered.

```
mysql> Salary DECIMAL(10,2)
-> DEFAULT 25000
```

Purpose

- Automatically assigns a valid default value.
- Reduces missing information.

Improved Employee Table

```
mysql> CREATE TABLE Employee
-> (
-> EmployeeID INT PRIMARY KEY,
-> EmployeeName VARCHAR(50) NOT NULL,
-> DepartmentID INT,
-> Salary DECIMAL(10,2) CHECK (Salary > 0),
-> Email VARCHAR(100) UNIQUE,
-> FOREIGN KEY (DepartmentID)
-> REFERENCES Department(DepartmentID)
-> );
```

Advantages of Integrity Constraints

- Prevents duplicate records.
- Eliminates invalid data entry.
- Maintains referential integrity.
- Improves data accuracy.
- Ensures consistency across related tables.
- Prevents insertion, update, and deletion anomalies.
- Simplifies database maintenance.

- Improves overall database reliability.

Justification

The proposed integrity constraints ensure that only valid and consistent data can be stored in the database. **Primary Key** guarantees uniqueness, **Foreign Key** maintains relationships between tables, **NOT NULL** ensures mandatory fields are filled, **UNIQUE** prevents duplicate values, **CHECK** validates business rules such as positive salary, and **DEFAULT** assigns predefined values when necessary. Together, these constraints eliminate inconsistencies and improve the quality and reliability of the database.

Conclusion

The original schema suffered from inconsistent data due to the absence of proper integrity constraints. By implementing **Primary Key, Foreign Key, NOT NULL, UNIQUE, CHECK, and DEFAULT** constraints, the database becomes more secure, accurate, and reliable. These constraints enforce valid data entry, preserve referential integrity, reduce anomalies, and ensure efficient database management. The improved schema is suitable for real-world database applications and satisfies the requirements of a well-designed relational database.

Q9. For a given relational schema, analyze the presence of join dependencies and decompose it into Fifth Normal Form (5NF) to remove redundancy.

Scenario Analysis

A company maintains information about **Suppliers, Products, and Projects** in a single relation. A supplier may supply multiple products, a product may be used in multiple projects, and a supplier may supply products for several projects. Storing all this information in one table creates **join dependencies**, resulting in data redundancy and maintenance problems.

The objective is to identify the join dependencies and decompose the relation into **Fifth Normal Form (5NF)** to eliminate redundancy while preserving all information.

Given Relation:

Supplier_Product_Project (SupplierID, ProductID, ProjectID)

Sample Data

SupplierID	ProductID	ProjectID
S1	P1	PR1
S1	P2	PR1
S2	P1	PR2
S2	P2	PR2

Analysis of Join Dependency

The relation has the following independent relationships:

- Supplier supplies Products.
- Supplier works with Projects.
- Product is used in Projects.

The relation contains a **Join Dependency (JD)** because the complete relation can be reconstructed by joining these smaller relations.

Join Dependency:

$(\text{SupplierID}, \text{ProductID}, \text{ProjectID}) \rightarrow (\text{SupplierID}, \text{ProductID})$

$(\text{SupplierID}, \text{ProjectID})$

$(\text{ProductID}, \text{ProjectID})$

Since the relation can be losslessly decomposed into smaller relations, it **does not satisfy Fifth**

Normal Form (5NF).

Problems in the Existing Relation

The original relation has the following issues:

- Data redundancy due to repeated combinations.
- Repeated supplier information.
- Repeated product information.
- Repeated project information.
- Update anomaly.

- Insertion anomaly.
- Deletion anomaly.
- High storage requirements.

Decomposition into Fifth Normal Form (5NF)

The relation is decomposed into three smaller relations.

Supplier_Product Table

SupplierID	ProductID
S1	P1
S1	P2
S2	P1
S2	P2

Primary Key: (SupplierID, ProductID)

Supplier_Project Table

SupplierID	ProjectID
S1	PR1
S2	PR2

Primary Key: (SupplierID, ProjectID)

Product_Project Table

ProductID	ProjectID
P1	PR1
P2	PR1
P1	PR2
P2	PR2

Primary Key: (ProductID, ProjectID)

Final 5NF Schema

Supplier_Product

(SupplierID **PK**, ProductID **PK**)

Supplier_Project

(SupplierID **PK**, ProjectID **PK**)

Product_Project

(ProductID **PK**, ProjectID **PK**)

Verification of 5NF

The decomposed relations satisfy **Fifth Normal Form (5NF)** because:

- Every join dependency is represented by candidate keys.
- No redundant tuples exist.
- The original relation can be reconstructed using natural joins without losing information.
- There are no remaining join anomalies.

Advantages of 5NF

- Eliminates join redundancy.
- Removes insertion, update, and deletion anomalies.
- Improves data consistency.
- Reduces unnecessary storage.
- Preserves lossless decomposition.
- Simplifies database maintenance.
- Supports efficient database design.
- Suitable for complex relational databases.

Justification

The original relation contained **join dependencies**, causing repeated combinations of suppliers, products, and projects. By decomposing the relation into **Supplier_Product**, **Supplier_Project**, and **Product_Project**, each table stores only one independent relationship. The original relation can be reconstructed through natural joins without losing any information. This decomposition eliminates redundancy, improves consistency, and satisfies the requirements of **Fifth Normal Form (5NF)**.

Conclusion

The given relational schema contained **join dependencies**, leading to redundancy and database anomalies. By decomposing the relation into **Supplier_Product**, **Supplier_Project**,

and **Product_Project** tables, the schema satisfies **Fifth Normal Form (5NF)**. The new design removes redundancy, preserves all information through lossless joins, improves data integrity, and provides a scalable and efficient database suitable for real-world applications.

Q10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

Scenario Analysis

A company maintains a database containing customer and order information. The existing SQL queries retrieve data inefficiently by using unnecessary subqueries, repeated table scans, and missing indexes. As the database grows, query execution becomes slow, increasing response time and reducing overall system performance.

The objective is to analyze the inefficient queries and optimize them using **JOIN operations, indexing techniques, and query restructuring** to improve retrieval speed and database performance.

Database Schema

Customer Table

CustomerID	CustomerName	City
C101	Arun	Chennai
C102	Priya	Vellore
C103	Rahul	Bangalore

Primary Key: CustomerID

Orders Table

OrderID	CustomerID	OrderDate	Amount
O101	C101	2026-08-01	5000
O102	C101	2026-08-02	3500
O103	C102	2026-08-03	7000
O104	C103	2026-08-04	4500

Foreign Key: CustomerID references Customer(CustomerID)

Inefficient Query

The following query retrieves customer names using a subquery.

```
mysql> SELECT CustomerName
-> FROM Customer
-> WHERE CustomerID IN
-> (
->     SELECT CustomerID
->     FROM Orders
-> );
```

```
+-----+
| CustomerName |
+-----+
| Arun         |
| Priya        |
| Rahul        |
+-----+
3 rows in set (0.02 sec)
```

Problems

- Uses a subquery unnecessarily.
- May perform multiple scans on large tables.
- Slower execution for large databases.
- Less efficient than using JOIN.

Optimized Query Using JOIN

```
mysql> SELECT DISTINCT
-> C.CustomerID,
-> C.CustomerName,
-> O.OrderID,
-> O.Amount
-> FROM Customer C
-> INNER JOIN Orders O
-> ON C.CustomerID = O.CustomerID;
```

```
+-----+-----+-----+-----+
| CustomerID | CustomerName | OrderID | Amount |
+-----+-----+-----+-----+
| C101       | Arun         | 0101    | 5000.00 |
| C101       | Arun         | 0102    | 3500.00 |
| C102       | Priya        | 0103    | 7000.00 |
| C103       | Rahul        | 0104    | 4500.00 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Explanation

- **INNER JOIN** directly combines Customer and Orders tables.
- Retrieves matching records efficiently.

- **DISTINCT** avoids duplicate rows.
- Improves execution speed compared to nested subqueries.

Creating an Index

To improve query performance, create an index on the foreign key.

```
mysql> CREATE INDEX idx_customer
      -> ON Orders(CustomerID);
Query OK, 0 rows affected (0.09 sec)
```

Benefits of Indexing

- Faster searching.
- Reduces table scanning.
- Improves JOIN performance.
- Enhances retrieval speed.

Query Restructuring

Instead of using multiple nested queries, use aggregation directly.

Inefficient Query

```
mysql> SELECT CustomerID
      -> FROM Orders
      -> GROUP BY CustomerID
      -> HAVING COUNT(*) > 1;
+-----+
| CustomerID |
+-----+
| C101       |
+-----+
1 row in set (0.02 sec)
```

Optimized Query

```
mysql> SELECT
      -> C.CustomerID,
      -> C.CustomerName,
      -> COUNT(O.OrderID) AS TotalOrders
      -> FROM Customer C
      -> JOIN Orders O
      -> ON C.CustomerID = O.CustomerID
      -> GROUP BY C.CustomerID, C.CustomerName;
+-----+-----+-----+
| CustomerID | CustomerName | TotalOrders |
+-----+-----+-----+
| C101       | Arun         | 2           |
| C102       | Priya        | 1           |
| C103       | Rahul        | 1           |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

This query retrieves customer details and the total number of orders in a single operation.

Performance Improvements

Technique	Improvement
INNER JOIN	Faster retrieval than nested subqueries
INDEX	Speeds up searching and JOIN operations
GROUP BY	Efficient aggregation of records
DISTINCT	Removes duplicate rows
Query Restructuring	Reduces unnecessary table scans

Advantages of Optimization

- Faster query execution.
- Reduces CPU and memory usage.
- Improves response time.
- Minimizes table scans.
- Efficient use of indexes.
- Better scalability for large databases.
- Improves overall database performance.
- Enhances user experience.

Justification

The optimized solution replaces inefficient subqueries with **JOIN** operations, which are more efficient for retrieving related data. Creating an **index** on the foreign key allows the database to locate matching records quickly, reducing search time. Query restructuring eliminates unnecessary operations and combines data retrieval into a single optimized query. These techniques significantly improve performance while maintaining data accuracy and consistency.

Conclusion

The original SQL queries were inefficient because they relied on unnecessary subqueries and lacked indexing. By optimizing the queries using **INNER JOIN**, **INDEX**, and **query restructuring**, the system achieves faster data retrieval, reduced execution time,

improved scalability, and better resource utilization. These optimization techniques are widely used in real-world database systems to ensure efficient and reliable performance.